Matteo Organek **ORG23613773**

Oleksi Ivanov **A00034498**

Obaida Muqbel **A00013570**

Nilay Oral **Z23607537**

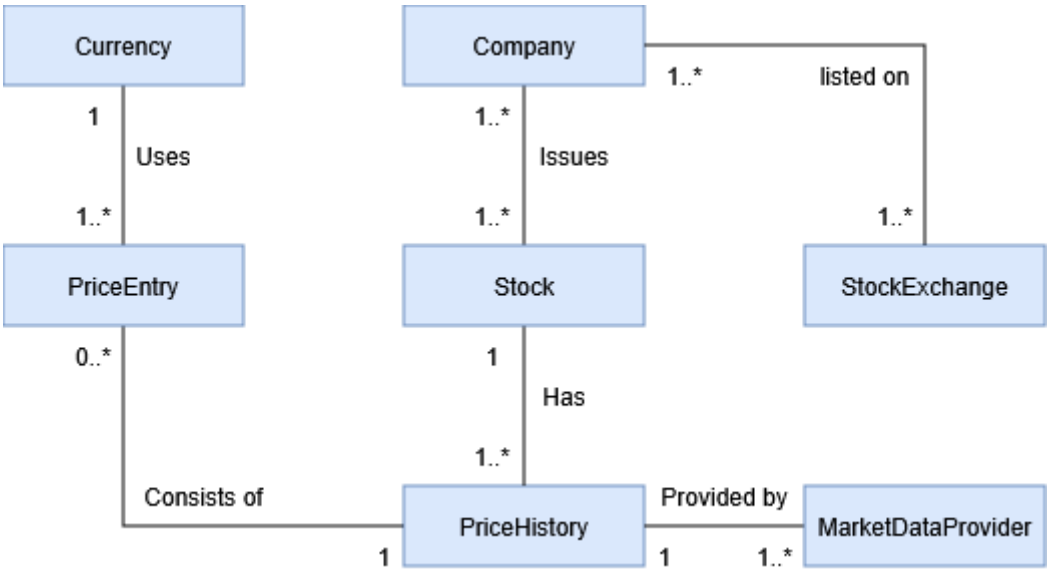
Architectural Design of StockComparer

This document describes the architectural design of the StockComparer system, a JavaFX based application for retrieving, storing, and comparing share prices over time. The architecture uses the Cheesman and Daniels component design process.

The process begins with a Business Concept Model to identify the business domain, followed by the Business Type Model, System and Business Interfaces, and ending in an Initial System Architecture. Each diagram is derived from the previous one, ensuring the final architecture is traceable back to the original requirements.

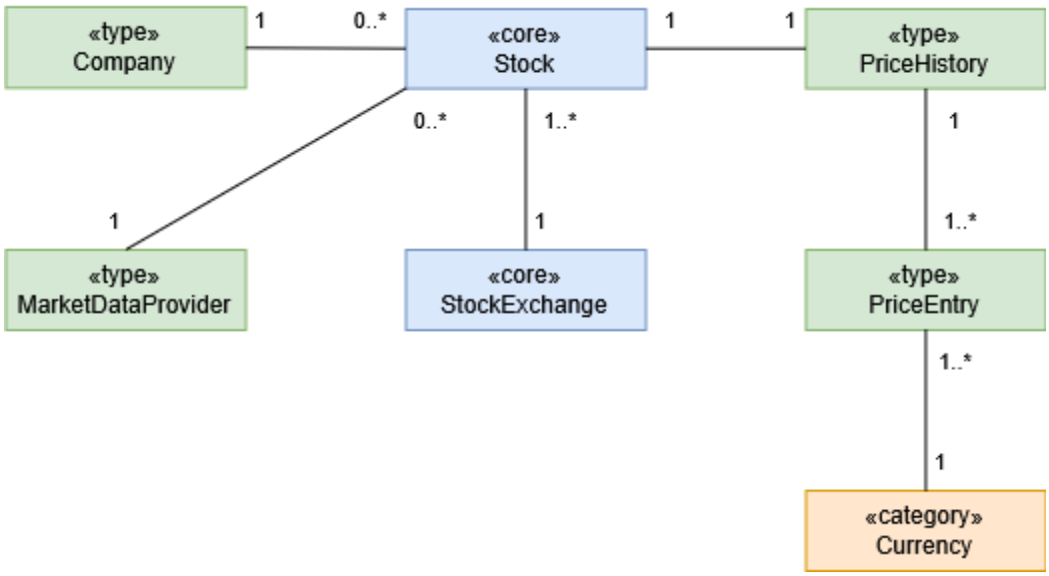
Business Concept Model

The Business Concept Model (BCM) was created with the business entities related to StockComparer in mind. Relationships between concepts were modelled as associations, with multiplicities capturing the cardinality constraints described in the requirements.



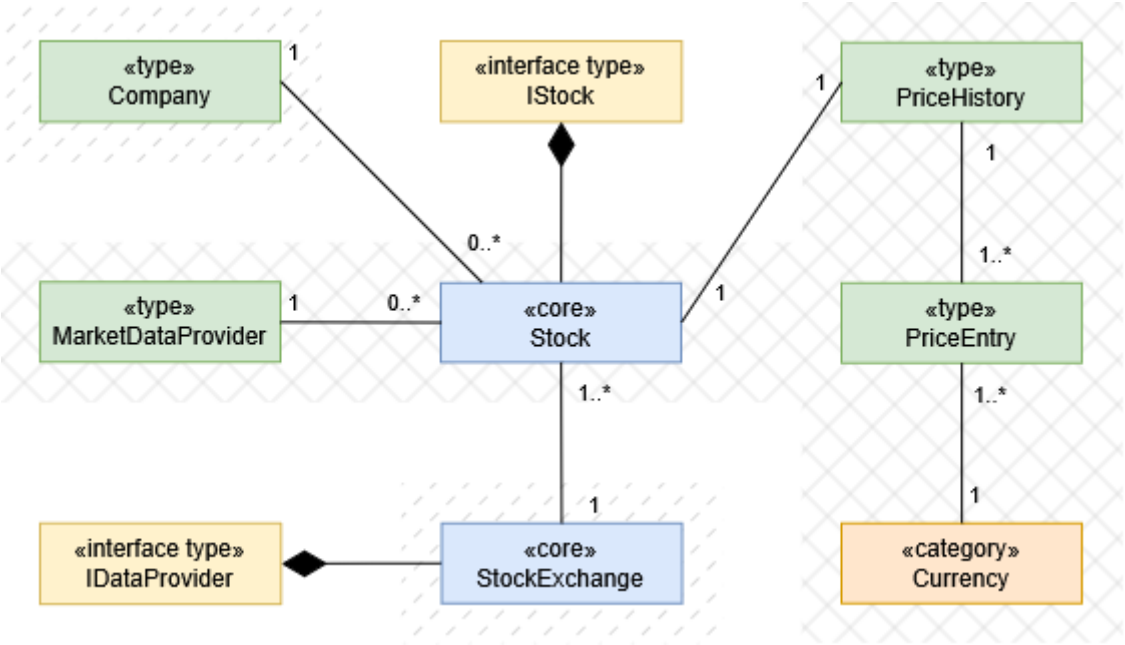
Business Type Model

The Business Type Model (BTM) is a refinement of the BCM. Its purpose is to scope the domain model to the specific application being built, classify each type according to its architectural significance, and identify responsibilities that will form the basis for component identification.



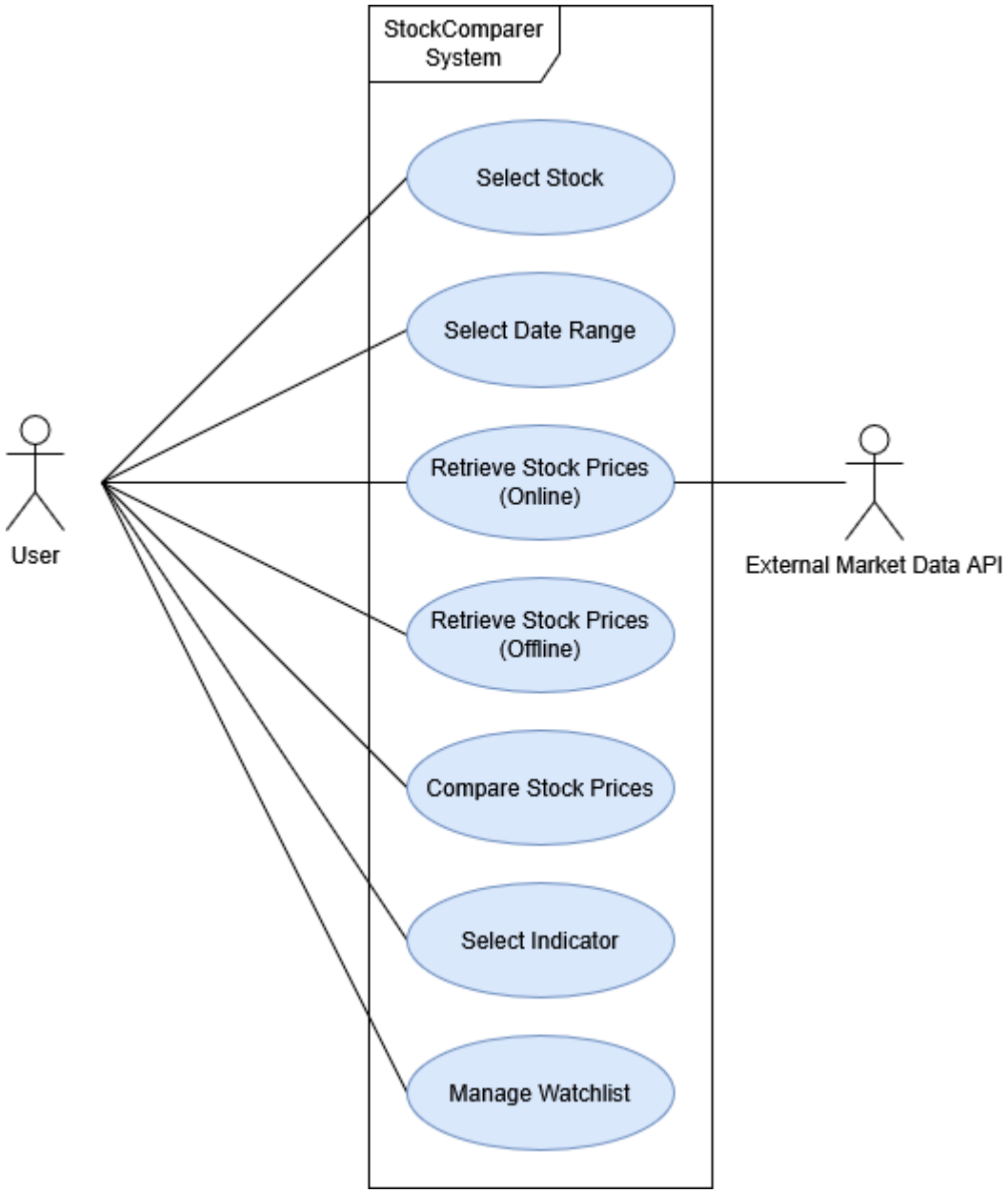
Business Interface

Business interfaces define the contractual boundary of each business component. They express what operations a component exposes to the rest of the system for managing the business entities within its sphere of responsibility. Business interfaces are defined at an abstract level, independent of any specific application use case.



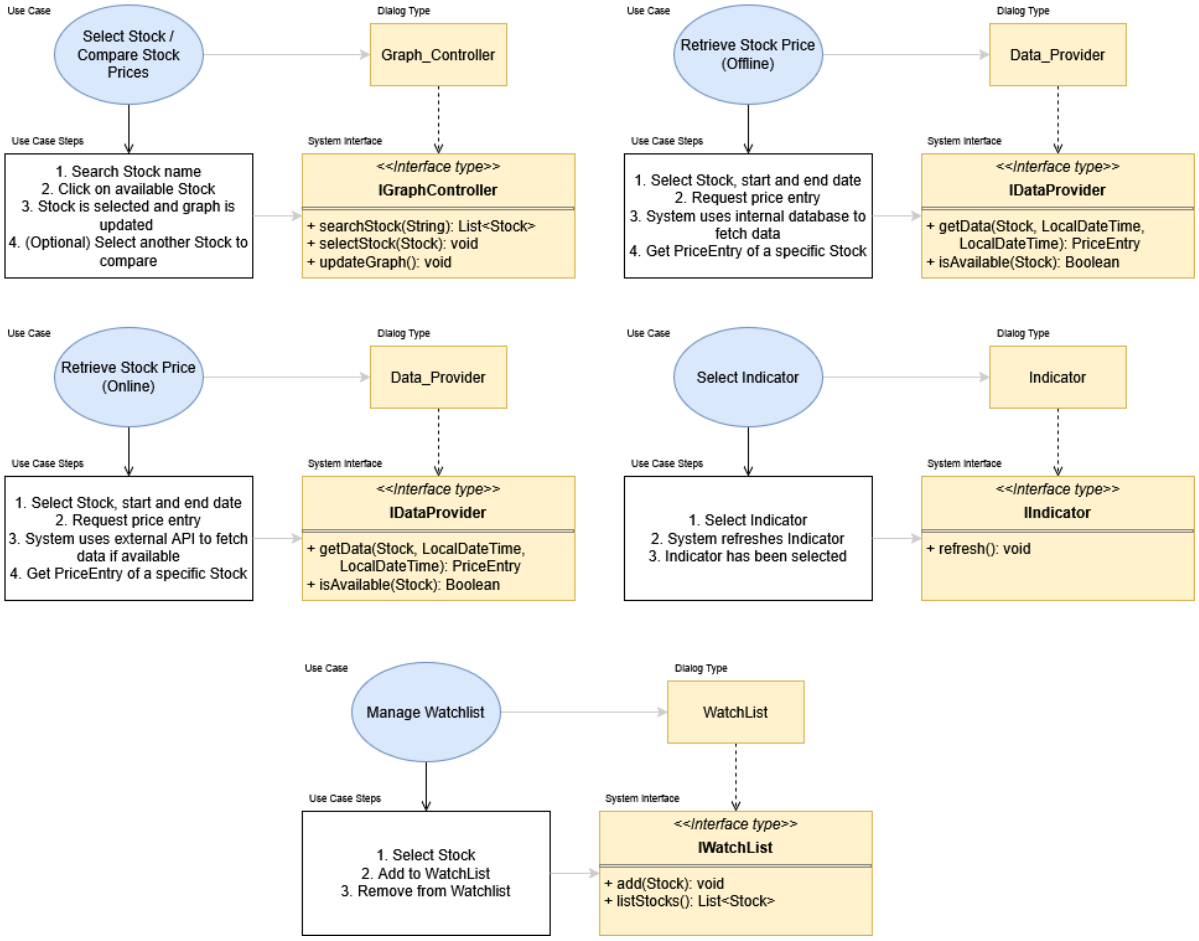
Use Case Diagram

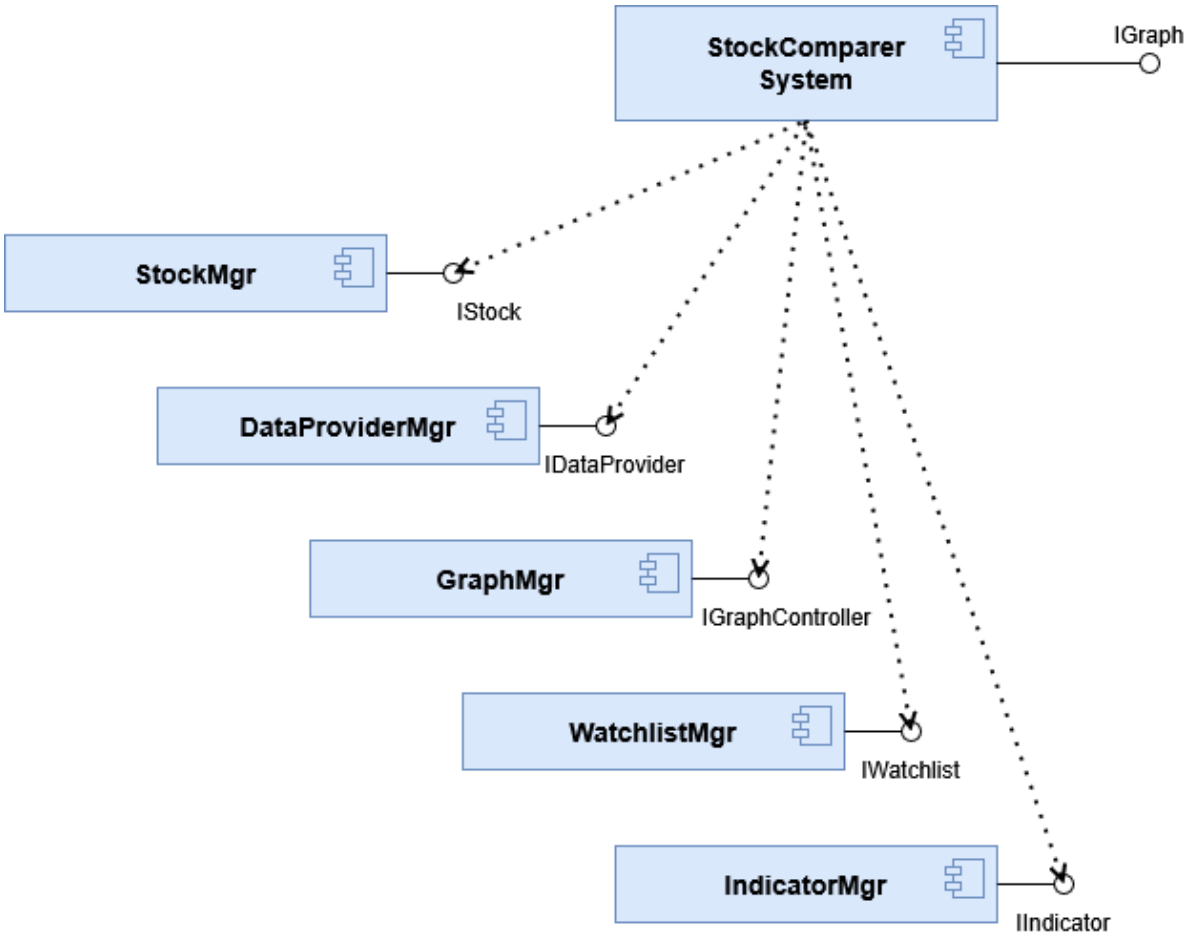
The use case model defines the functional requirements of the system from the perspective of its actors. Each use case represents a sequence of interactions between an actor and the system that delivers a meaningful outcome. The use case model drives the identification of system interfaces: each use case leads to a system interface with operations corresponding to the system steps of that use case.



System Interfaces and Operations

System interfaces (SI) define the operations that the system component must expose to support the dialog layer (the controllers and presenters). One system interface is defined per use case, containing only the operations required to implement the system steps of that use case. Operations that require querying or mutating system state are included; operations that can be handled purely within the dialog layer using cached data are excluded.



Initial System Architecture

The Initial System Architecture diagram (ISA) summarises the component decomposition of the system, showing which components exist, which interfaces they provide, and how the system component depends on the business components. It is the primary deliverable of the component identification phase and forms the blueprint for subsequent implementation.